

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM ENGENHARIA DE SOFTWARE

FABRÍCIO ROCHA DE SOUZA
GUILHERME AIRES PIMENTA DE MACEDO
LUIZ EDUARDO DIAS
MARIANA DA ROCHA PEREIRA MOREIRA

SISTEMA DE VALIDAÇÃO DE IMAGENS E AUTENTICAÇÃO SEGURA

Mogi das Cruzes, SP

2026

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM ENGENHARIA DE SOFTWARE

FABRÍCIO ROCHA DE SOUZA
GUILHERME AIRES PIMENTA DE MACEDO
LUIZ EDUARDO DIAS
MARIANA DA ROCHA PEREIRA MOREIRA

SISTEMA DE VALIDAÇÃO DE IMAGENS E AUTENTICAÇÃO SEGURA

Projeto de conclusão de disciplina apresentado
ao curso de Engenharia de software da
Universidade de Mogi das Cruzes.

Prof. Orientador: Fabiano Bezerra Menegidio

Mogi das Cruzes, SP

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema web voltado à análise de imagens com o objetivo de identificar conteúdos potencialmente gerados por inteligência artificial, aliado a um robusto mecanismo de autenticação e segurança da informação. O sistema implementa autenticação em dois fatores (2FA), controle de acesso, auditoria, controle de acesso criptografado e foco na segurança.

Palavras-chave: segurança; autenticação; 2FA; LGPD.

ABSTRACT

This work presents the development of a web system focused on image analysis, aiming to identify content potentially generated by artificial intelligence, combined with a robust authentication and information security mechanism. The system implements two-factor authentication (2FA), access control, auditing, encrypted access control, and a strong focus on security.

Keywords: security; authentication; 2FA; LGPD.

LISTA DE ILUSTRAÇÕES

Figura 1: Diagrama - Apresenta o diagrama de arquitetura do projeto.

Figura 2: Apresenta tela de login.

Figura 3: Apresenta tela de cadastro com senha válida.

Figura 4: Apresenta tela após logar com sucesso.

Figura 5: Apresenta tela para análise de imagens.

Figura 6: Apresenta tela com QR Code de ativação do 2FA.

Figura 7: Apresenta tela para controle dos dados.

SUMÁRIO

INTRODUÇÃO	10
1. VISÃO GERAL DO SISTEMA	11
1.1 Requisitos Funcionais de Segurança (RF-SEG)	11
1.2 Requisitos Não-Funcionais de Segurança e Privacidade (RNF-SEG)	12
2. ARQUITETURA DO SISTEMA	13
2.1 Visão geral da Arquitetura	13
2.2 Stack Tecnológico	13
2.3 Diagrama de Arquitetura	14
3. FLUXOS DE AUTENTICAÇÃO E DADOS DOCUMENTADOS	15
3.1 Registro de Novo Usuário	15
3.2 Login	15
3.3 Fluxo de 2FA	16
3.4 Renovação de Tokens (Refresh)	16
3.5 Logout	16
3.6 Recuperação de senha	16
4. AUTENTICAÇÃO E GESTÃO DE CREDENCIAIS	20
4.1 Hash de Senhas — bcrypt com 12 rounds	20
4.2 Gestão de Sessões — Dupla Camada JWT	20
4.3 Blacklist de Access Tokens	21
4.4 Proteção contra Força Bruta	21
4.5 Validação de credenciais	21
4.6 Autenticação de Dois Fatores (2FA)	21
5. CRIPTOGRAFIA	22

5.1 Estratégia de Criptografia em Camadas	22
5.2 AES-256-CBC para Dados em Repouso	22
5.3 HTTPS/TLS Obrigatório	23
5.4 Justificativas das Escolhas Criptográficas.....	23
5.4.1 Bcrypt	23
5.4.2 AES-256-CBC vs. AES-256-GCM	23
5.4.3 SHA-256 para token de recuperação	23
5.4.4 HS256 para JWT	23
6. ATIVOS DO SISTEMA	24
7. IDENTIFICAÇÃO DE AMEAÇAS E VULNERABILIDADES	24
7.1 Spoofing - Falsificação de Identidade	24
7.1.1 - Força bruta no login	24
7.1.2 - Reutilização de token JWT após logout	24
7.1.3 - Falsificação do token intermediário do 2FA	24
7.2 Tampering - Adulteração de Dados	24
7.2.1 - Interceptação de dados em trânsito	24
7.2.2 - Adulteração do payload JWT	25
7.2.3 - Injeção via campo de imagem	25
7.3 Repudiation - Repúdio	25
7.3.1 - Ausência de rastreabilidade de ações	25
7.4 Information Disclosure - Vazamento de Informação	25
7.4.1 - Retorno de dados sensíveis para o cliente	25
7.4.2 - Dados sensíveis armazenados sem criptografia	25
7.4.3 - API Key do Gemini exposta	25
7.5 Denial of Service - Negação de Serviço	25
7.5.1 - Flood de requisições no login	25
7.5.2 - Exaustão da cota da API Gemini	25
7.6 Elevation of Privilege - Elevação de Privilégio	26

7.6.1 - Acesso a rotas protegidas sem autenticação	26
7.6.2 - Token forjado com ID de outro usuário	26
8. ASSOCIAÇÃO RISCO × CONTRAMEDIDA	26
9. TESTES DE SEGURANÇA REALIZADOS	27
9.1 Testes Automatizados	27
9.1.1 - Verificação de saúde da API	27
9.1.2 - Rejeição de credenciais inválidas	28
9.1.3 - Renderização do formulário de login	28
9.2 Testes Manuais	28
9.2.1 - Bloqueio por força bruta	28
9.2.2 - Acesso a rota protegida sem token	29
9.2.3 - Uso de token após logout	29
9.2.4 - Cadastro com senha fraca	29
9.2.5 - Rate limiting na análise de imagem	29
9.2.6 - Fluxo 2FA com token expirado	29
9.2.7 - Injeção NoSQL no campo de e-mail	30
9.2.8 - Exportação de dados sem campos sensíveis	30
10. RESULTADOS DOS TESTES DOCUMENTADOS	30
11. CONFORMIDADE COM A LGPD	31
11.1 Dados pessoais coletados	31
11.2 Funcionalidades LGPD Implementadas	31
11.3 Minimização de Dados	32
12. AUDITORIA E LOGS	32
12.1 Arquitetura da Trilha de Auditoria	32
12.2 Catálogo de Ações Auditadas	32
12.3 Exemplo de Análise de Logs	33
12.3.1 LOGIN_FAILED	34
12.3.2 LOGIN_SUCCESS.....	34

12.3.3 2FA	35
12.3.4 BUSCA POR EMAIL	35
12.4 Proteção contra alteração de logs	36
13. NORMAS TÉCNICAS	36

INTRODUÇÃO

O avanço acelerado da inteligência artificial tem possibilitado a criação de conteúdos digitais cada vez sofisticados. Modelos baseados em técnicas como Generative Adversarial Networks (GANs) e outras abordagens têm sido amplamente utilizadas para produzir imagens com o mais alto grau de realismo, tornando cada vez mais difícil para as pessoas distinguirem conteúdos autênticos daqueles gerados artificialmente.

Nesse contexto, surgem grandes preocupações relacionadas à confiabilidade da informação, à disseminação de desinformação e ao uso indevido dessas tecnologias, tornando-se essencial o desenvolvimento de ferramentas capazes de auxiliar na identificação de conteúdos gerados por inteligência artificial. Paralelamente, o tratamento de dados exige extrema atenção às práticas de segurança da informação, especialmente no que diz respeito à proteção de dados pessoais, conforme estabelecido pela Lei Geral de Proteção de Dados Pessoais (LGPD).

Dessa forma, este trabalho tem como objetivo o desenvolvimento de uma aplicação web voltada à validação de imagens, capaz de indicar a probabilidade de um determinado conteúdo ter sido gerado por inteligência artificial, aliada à implementação de um conjunto de práticas e mecanismos de segurança da informação. O sistema proposto busca não apenas oferecer uma solução funcional, mas também atender a requisitos fundamentais de confidencialidade, integridade e disponibilidade dos dados.

1. Visão Geral do Sistema

O sistema proposto consiste em uma aplicação web voltada para a análise de imagens com o objetivo de identificar a probabilidade de terem sido geradas por inteligência artificial. A plataforma permite que os usuários autenticados realizem upload de imagens, as quais são processadas por um modelo de análise, retornando um índice probabilístico de geração por IA.

Além de sua funcionalidade principal, o sistema incorpora mecanismos de segurança da informação, incluindo autenticação de usuários, controle de acesso, recuperação de senha e proteção de dados sensíveis. A arquitetura foi projetada considerando os princípios da confidencialidade, integridade e disponibilidade (CIA), garantindo a proteção dos dados dos usuários e a confiabilidade do sistema.

O público-alvo inclui estudantes, pesquisadores e profissionais interessados em validar o conteúdo que circula por meio digital, seu acesso será via navegador web.

1.1 Requisitos Funcionais de Segurança (RF-SEG)

RF-SEG-01 (Controle de Acesso Baseado em Funções – RBAC): O sistema deve implementar controle de acesso segregado, garantindo que “Usuários Comuns” não tenham acesso às interfaces e rotas exclusivas de “Verificadores” ou “Administradores”.

RF-SEG-02 (Escaneamento de Malware em Uploads): O sistema deve verificar imagens enviadas para análise em busca de malwares antes de armazená-los no servidor.

RF-SEG-03 (Trilha de Custódia de Mídia): O sistema deve registrar um log imutável de todas as interações com uma mídia sob investigação. Deve ficar registrado quem baixou o arquivo, quem alterou o status de "em análise" para "fake news" e quando isso ocorreu, garantindo a integridade do laudo.

RF-SEG-04 (Gestão Avançada de Sessão): O sistema deve fornecer uma interface onde o usuário possa visualizar todos os dispositivos atualmente conectados à sua conta e permitir o “logout remoto” de sessões específicas.

RF-SEG-05 (Mascaramento de Dados – Data Masking): O sistema deve ofuscar informações sensíveis dos usuários (como e-mail parcial e telefone) nas telas de listagem administrativa, exibindo os dados completos apenas sob demanda e com registro de auditoria.

RF-SEG-06 (Anonimização de Relatórios): O sistema deve garantir que os laudos e estatísticas públicas geradas sobre desinformação sejam totalmente anonimizados, impedindo a reidentificação dos usuários que enviaram as mídias originais.

1.2 Requisitos Não-Funcionais de Segurança e Privacidade (RNF-SEG)

RNF-SEG-01 (Política de Retenção e Descarte Automático): O sistema deve implementar uma rotina automatizada que exclua de forma irrecuperável (data wiping) arquivos de mídia e dados pessoais de contas inativas após 5 anos, em alinhamento aos princípios da LGPD.

RNF-SEG-02 (Defesa contra XSS e Injeções): A aplicação web deve implementar cabeçalhos de segurança estritos, incluindo Content Security Policy (CSP), para mitigar ataques de Cross-Site Scripting (XSS) e injeção de código.

RNF-SEG-03 (Vinculação de Sessão - Session Binding): Para evitar que um invasor roube o cookie de sessão de um usuário e acesse a conta de outro computador (ataque de Session Hijacking), o sistema deve invalidar a sessão imediatamente se detectar uma mudança drástica no endereço IP ou no navegador (User-Agent).

RNF-SEG-04 (Segregação de Privilégios no Banco de Dados): A aplicação deve conectar-se ao banco de dados utilizando um usuário com privilégios restritos (apenas DML – Data Manipulation Language), sem permissões para alterar a estrutura do banco (DDL).

RNF-SEG-05 (Rate Limiting de Serviços de Negócio): Além das rotas de autenticação, as APIs de envio de mídia para análise devem possuir limitação de requisições por minuto por IP (Rate Limit), prevenindo ataques de Negação de Serviço (DoS) e esgotamento de recursos.

RNF-SEG-06 (Proteção de Formulários Abertos): Todas as rotas públicas de submissão de links ou mídias devem implementar desafios anti-bot (como um CAPTCHA invisível) para evitar ataques de automação, spam ou exaustão de recursos.

2. Arquitetura do sistema

2.1 Visão Geral da Arquitetura

O sistema adota uma arquitetura cliente-servidor desacoplada, com frontend e backend deployados em plataformas de nuvem independentes. Toda a comunicação entre as camadas é realizada via HTTPS, garantindo confidencialidade em trânsito.

2.2 Stack Tecnológico

Camada	Tecnologia	Versão	Função
Frontend	React	19.2	Interface do usuário (SPA)
Frontend	TypeScript	6.0	Tipagem estática
Frontend	Vite	8.0	Build e bundling
Frontend	Tailwind CSS	3.4	Estilização
Frontend	Axios	1.15	Cliente HTTP com interceptors
Frontend	React Router	7.14	Roteamento SPA
Backend	Node.js / Express	5.2	API REST
Backend	TypeScript	5.8	Tipagem estática
Backend	Mongoose	9.6	ODM para o MongoDB
Backend	bcrypt	6.0	Hash de senhas
Backend	jsonwebtoken	9.0	Geração e verificação de JWT
Backend	speakeasy	2.0	TOTP para 2FA

Backend	express-rate-limit	8.3	Rate limiting
Backend	@google/generative-ai	0.24	Integração com Gemini
Backend	nodemailer	8.0	Envio de e-mail
Banco de Dados	MongoDB Atlas	—	Persistência NoSQL
Hospedagem Frontend	Vercel	—	CDN + HTTPS automático
Hospedagem Backend	Railway	—	Container + HTTPS automático
IA	Google Gemini Flash	—	Análise de imagens

2.3 Diagrama de Arquitetura

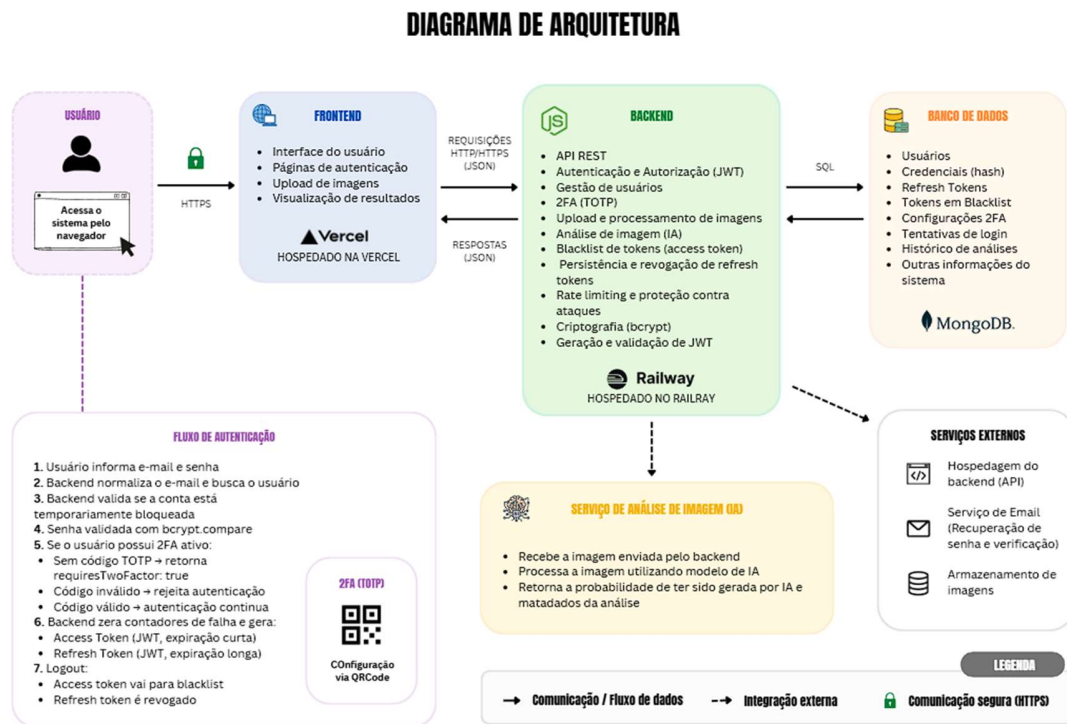


Figura 1: Apresenta o diagrama de arquitetura do projeto.

3. Fluxos de autenticação e dados documentados

O fluxo de autenticação do sistema foi projetado com foco em segurança, controle de acesso e mitigação de ataques automatizados, seguindo os itens abaixo:

3.1 Registro de Novo Usuário

1. Usuário envia e-mail, senha e flag de consentimento (consent: true).
2. Backend valida formato do e-mail (regex) e força da senha (mínimo 8 caracteres, maiúscula, minúscula, número e caractere especial).
3. Verificação de unicidade: se o e-mail já existe, retorna erro 400.
4. Senha é hasheada com bcrypt (12 rounds). O hash gerado inclui o salt embutido.
5. Usuário é persistido no MongoDB com consent: true e consentDate.
6. Log de auditoria é registrado: REGISTER_SUCCESS ou REGISTER_FAILED com motivo.
7. Retorno: HTTP 201 com id e e-mail do usuário criado.

3.2 Login

1. Usuário envia e-mail e senha.
2. Backend normaliza o e-mail (trim + lowercase) e busca o usuário.
3. Atraso fixo de 700ms é aplicado independentemente do resultado (mitiga timing attacks).
4. Verificação de bloqueio de conta: se locked_until > now, retorna erro.
5. bcrypt.compare valida a senha contra o hash armazenado.
6. Se a senha for inválida: incrementa failed_login_attempts. Após 5 tentativas, define locked_until = now + 15 minutos.
7. Se 2FA estiver ativo: retorna requiresTwoFactor: true e um twoFactorToken JWT de 5 minutos (não completa o login ainda).
8. Se 2FA não estiver ativo: gera accessToken (15min) e refreshToken (7d), persiste o refresh token no banco, retorna os tokens.
9. Logs registrados: LOGIN_SUCCESS, LOGIN_FAILED ou LOGIN_2FA_REQUIRED.

3.3 Fluxo de 2FA

1. Usuário recebe o twoFactorToken e insere o código TOTP do aplicativo autenticador.
2. Backend verifica a assinatura e propósito do twoFactorToken (purpose: "2fa").
3. Código TOTP é validado via speakeasy com janela de tolerância de 2 intervalos (± 60 segundos).
4. Se inválido: incrementa failed_login_attempts e retorna erro.
5. Se válido: gera os tokens de sessão completos e finaliza o login.

3.4 Renovação de Tokens (Refresh)

8. Frontend intercepta respostas 401 automaticamente (axios interceptor).
9. Envia o refreshToken para POST /api/auth/refresh.
10. Backend verifica assinatura e validade do refresh token no banco.
11. Token antigo é revogado (revoked: true). Novo access token e refresh token são gerados e persistidos.
12. Frontend atualiza o localStorage e reenvia a requisição original.

3.5 Logout

1. Frontend envia o refreshToken no corpo da requisição.
2. Access token é inserido na blacklist com sua expiração original.
3. Refresh token é marcado como revogado no banco.
4. Frontend limpa o localStorage e redireciona para a tela de login.

3.6 Recuperação de Senha

1. Usuário solicita recuperação informando o e-mail.
2. Backend retorna sempre a mesma mensagem de sucesso, independentemente de o e-mail existir (evita enumeração de usuários).
3. Se o e-mail existir: gera 32 bytes aleatórios (rawToken), armazena apenas o hash SHA-256 (hashedToken) no banco com expiração de 15 minutos.
4. E-mail é enviado via nodemailer (Gmail SMTP) com link contendo o rawToken.
5. Usuário clica no link e envia rawToken + nova senha.
6. Backend recalcula o hash SHA-256 e compara com o armazenado.

7. Se válido e não expirado: nova senha é hasheada com bcrypt e salva. Token é limpo.
Todos os refresh tokens do usuário são revogados.

No que se refere ao fluxo de dados, o sistema segue uma arquitetura segura na qual todas as requisições autenticadas dependem da validação do token JWT. O envio de imagens para análise ocorre apenas após autenticação válida, sendo o processamento realizado pelo backend e o resultado retornado ao usuário de forma segura.

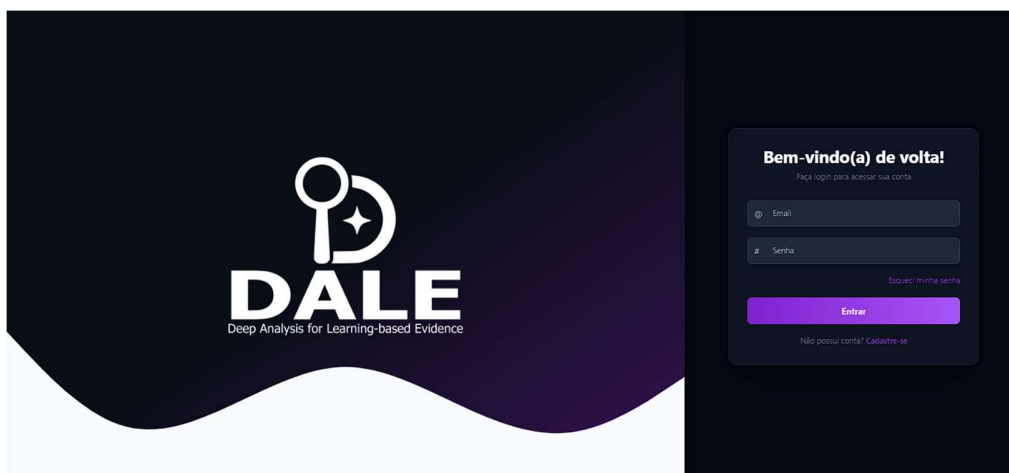


Figura 2: Apresenta tela de login.

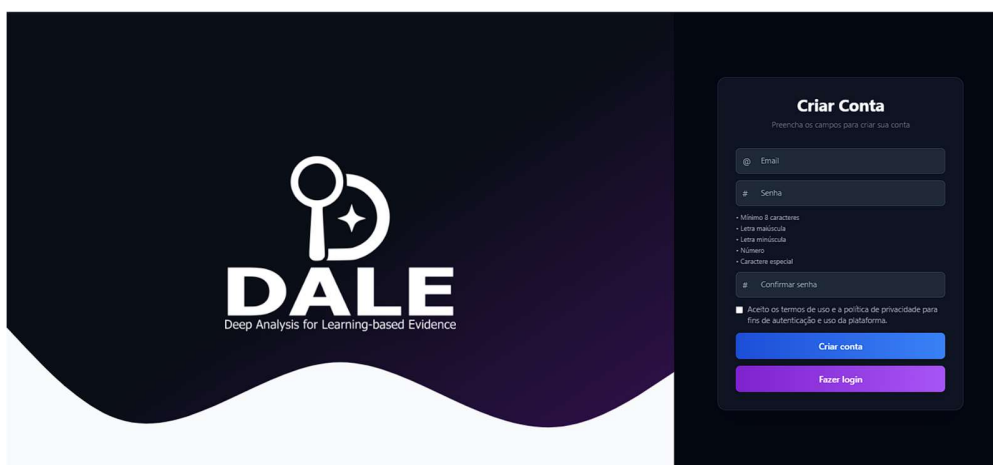


Figura 3: Apresenta tela de cadastro com senha válida.

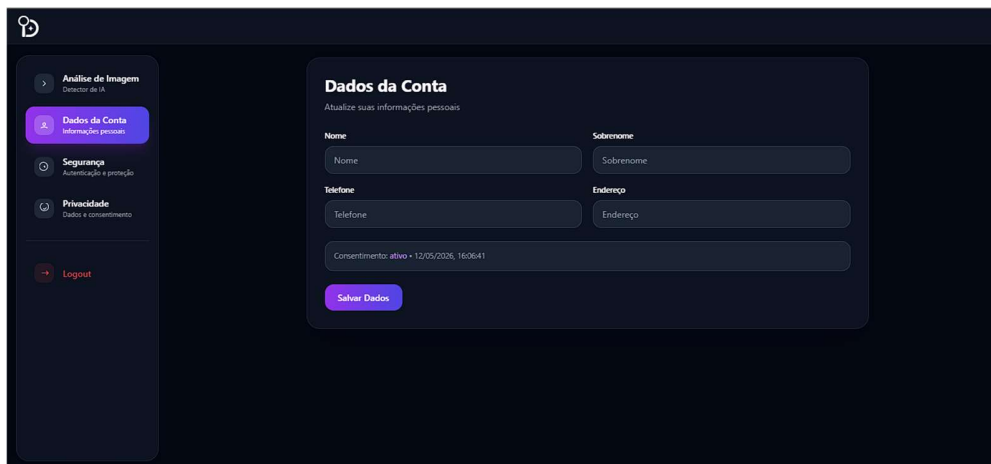


Figura 4: Apresenta tela após login com sucesso.

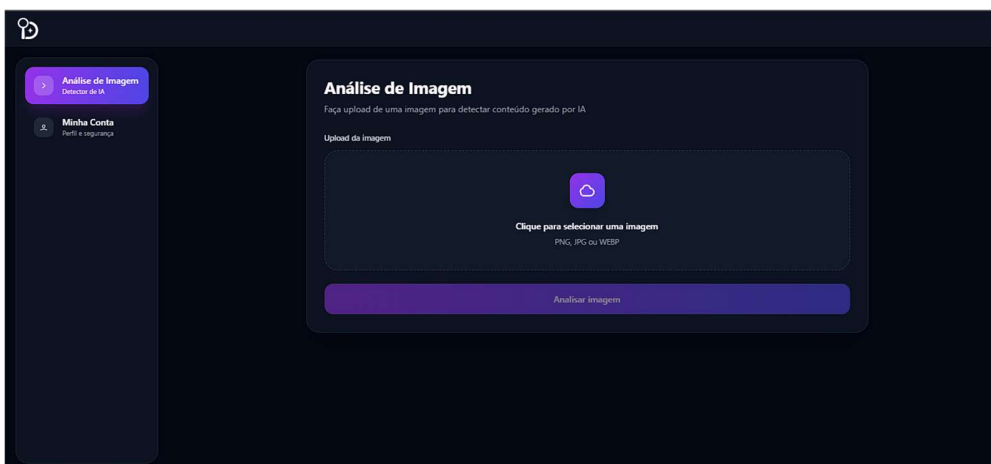


Figura 5: Apresenta tela para análise de imagens.

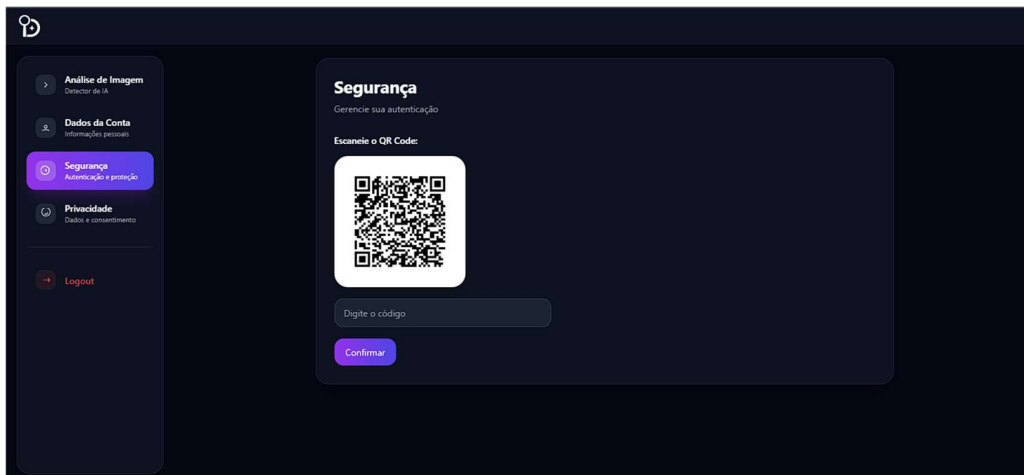


Figura 6: Apresenta tela com QR Code de ativação do 2FA.

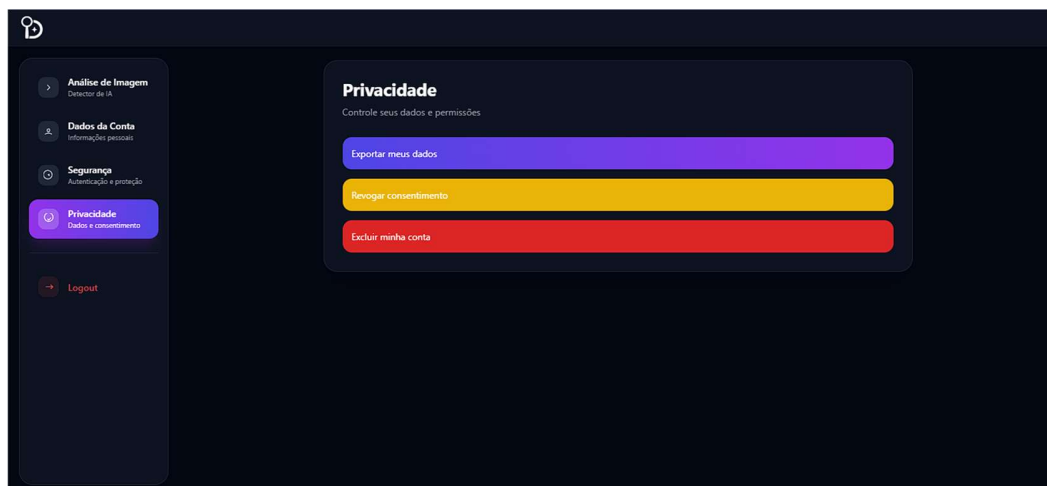


Figura 7: Apresenta tela para controle dos dados.

4. Autenticação e Gestão de credenciais

4.1 Hash de Senhas — bcrypt com 12 rounds

Senhas nunca são armazenadas em texto claro. O bcrypt foi escolhido por ser um algoritmo de hashing adaptativo projetado especificamente para senhas, diferente de SHA-256 ou MD5 que são funções de hash genéricas e rápidas. Com 12 rounds, o tempo de hashing é de aproximadamente 200-400ms em hardware moderno, suficiente para tornar ataques de força bruta inviáveis sem impactar a experiência do usuário.

O bcrypt incorpora salt único e aleatório por usuário de forma nativa, eliminando ataques de rainbow table. O hash resultante (60 caracteres) inclui o salt embutido, simplificando a verificação.

```
// auth.service.ts - Geração
const hash = await bcrypt.hash(password, env.BCRYPT_ROUNDS) // 12
rounds
// auth.service.ts - Verificação
const valid = await bcrypt.compare(password || '', user.password || '')
```

4.2 Gestão de Sessões — Dupla Camada JWT

O sistema usa dois tokens JWT com propósitos distintos, assinados com segredos separados:

Token	Duração	Segredo	Propósito
Access Token	15 minutos	JWT_SECRET	Autenticação de requisições
Refresh Token	7 dias	JWT_REFRESH_SECRET	Renovação de sessão (persistido no BD)
2FA Token	5 minutos	JWT_SECRET (purpose: "2fa")	Etapa intermediária do fluxo 2FA

Essa separação impede que um refresh token seja aceito como access token. A renovação de tokens implementa rotação: o refresh token antigo é revogado a cada renovação, impedindo reutilização.

4.3 Blacklist de Access Tokens

Como JWTs são stateless por natureza, sua invalidação antes do vencimento requer mecanismo explícito. NO logout, o access token é inserido na coleção TokenBlacklist com sua data de expiração original. Todas as requisições autenticadas consultam a blacklist antes de autorizar o acesso.

A limpeza automática de tokens expirados na blacklist evita crescimento indefinido da coleção (método `cleanupExpired` da `BlackListRepository`).

4.4 Proteção contra Força Bruta

Três camadas complementares de proteção:

- Bloqueio de conta: após 5 tentativas inválidas, a conta é bloqueada por 15 minutos (campo `locked_until`). O bloqueio é verificado antes da comparação de senha.
- Atraso fixo: 700 ms de espera aplicados em todos os fluxos de autenticação (credenciais e 2FA), dificultando ataques de enumeração por tempo de resposta.
- Rate limiting de rota: `POST/api/auth/login` e `/api/auth/refresh` aceitam no máximo 5 requisições falhas por IP a cada 15 minutos (`skipSuccessfulRequests: true`).

4.5 Validação de Credenciais

As regras de validação são aplicadas no backend, independentemente do frontend:

- E-mail: validação por regex (`/^[^s@]+@[^s@]+\.[^s@]+$/`).
- Senha forte: mínimo 8 caracteres, com pelo menos uma maiúscula, uma minúscula, um dígito e um caractere especial.
- E-mail normalizado (trim + lowercase) antes de toda busca no banco.

4.6 Autenticação de Dois Fatores (2FA)

- Protocolo: TOTP (RFC 6238)

- O 2FA usa TOTP, gerando códigos de uso único baseados no tempo e em um segredo criptográfico único por usuário. O segredo é gerado com `speakeasy.generateSecret()` e armazenado em base32. A ativação exige confirmação com um código válido antes de habilitar o 2FA na conta.
- A verificação aceita uma janela de tolerância de 2 intervalos (± 60 segundos) para compensar possíveis diferenças de relógio entre o dispositivo do usuário e o servidor.

5. Criptografia

5.1 Estratégia de Criptografia em Camadas

O sistema implementa criptografia em quatro contextos distintos:

Contexto	Algoritmo	Parâmetros	Localização
Hash de senhas	bcrypt	12 rounds, salt automático	auth.service.ts
Dados sensíveis (perfil)	AES-256-CBC	IV aleatório 128 bits por operação	shared/utils/crypto.ts
Token de recuperação	SHA-256	32 bytes de entropia	auth.service.ts
Dados em trânsito	TLS 1.2+	HTTPS obrigatório em produção	app.ts + Vercel/ Railway
Tokens de sessão	HS256 (HMAC-SHA256)	Segredos distintos por tipo	token.service.ts
2FA	TOTP (RFC 6238)	Segredo base32, janela ± 2	security/two-factor.service.ts

5.2 AES-256-CBC para Dados em Repouso

Dados sensíveis do perfil do usuário são criptografados com AES-256-CBC antes de serem armazenadas no MongoDB. A chave de 256 bits é derivada da variável de ambiente `SECRET_KEY` via SHA-256. Um IV de 16 bytes é gerado aleatoriamente por operação e armazenado junto ao ciphertext no formato `iv_hex:encrypted_hex`.

5.3 HTTPS/TLS Obrigatório

Em ambiente de produção, o backend redireciona automaticamente requisições HTTP para HTTPS. As plataformas de hospedagem (Vercel e RailWay) gerenciam os certificados TLS automaticamente e forçam HTTPS em todos os acessos.

5.4 Justificativas das Escolhas Criptográficas

5.4.1 bcrypt

O bcrypt foi projetado para ser lento e adaptativo e , permanece fortemente recomendado pela OWASP para Node.js e apresenta suporte maduro e auditado. Com 12 rounds, o bcrypt representa equilíbrio adequado entre segurança e desempenho para o contexto deste projeto.

5.4.2 AES-256-CBC vs. AES-256-GCM

AES-256-CBC com IV aleatório é seguro para confidencialidade de dados em repouso. Para futuras versões do sistema, a migração para GCM é recomendada, pois detecta adulterações no dado cifrado.

5.4.3 SHA-256 para token de recuperação

O token de reset é um valor aleatório de 256 bits de entropia, não sujeito a ataques de dicionário. SHA-256 é adequado para este caso de uso, pois o objetivo é não armazenar o token bruto no banco. Mesmo que o banco seja comprometido, o atacante não consegue derivar o token original do hash sem ter o valor gerado, tornando os tokens de reset inutilizáveis fora da janela de 15 minutos.

5.4.4 HS256 para JWT

HS256 (HMAC-SHA256) usa chave simétrica, adequado para um sistema onde frontend e backend compartilham o mesmo ambiente de segredo. No contexto deste projeto (único backend responsável por emissão e verificação), HS256 é suficiente e mais eficiente computacionalmente.

6. Ativos do Sistema

Os ativos foram classificados de acordo com sua importância para a operação do sistema:

- **Ativos de Dados:** Banco de dados MongoDB, contendo credenciais (hashes), tokens em blacklist e as configurações de 2FA.
- **Ativos de Software:** Código-fonte do backend em Node.js e interface React frontend.
- **Ativos de Infraestrutura:** Instâncias de hospedagem na Vercel e Railway.
- **Ativos de Identidade:** Segredos de 2FA e chaves privadas de assinatura de tokens JWT.

7. Identificação de Ameaças e Vulnerabilidades

Para identificar as ameaças do sistema, utilizamos a metodologia STRIDE, que categoriza os principais tipos de ataques em aplicações web. A análise foi feita com base no código desenvolvido e nas funcionalidades implementadas.

7.1 Spoofing - Falsificação de Identidade

7.1.1 - Força bruta no login

Um atacante pode tentar adivinhar senhas de usuários fazendo muitas tentativas seguidas no endpoint de login.

7.1.2 - Reutilização de token JWT após logout

Se um token roubado não fosse invalidado, poderia ser reutilizado mesmo após o usuário sair da conta.

7.1.3 - Falsificação do token intermediário do 2FA

No fluxo de dois fatores, um token temporário é gerado. Sem validação adequada, poderia ser forjado.

7.2 Tampering - Adulteração de Dados

7.2.1 - Interceptação de dados em trânsito

Sem HTTPS, um atacante na mesma rede poderia interceptar e modificar requisições.

7.2.2 - Adulteração do payload JWT

Tokens JWT sem assinatura forte poderiam ter seu conteúdo alterado para se passar por outro usuário.

7.2.3 - Injeção via campo de imagem

O campo `imageBase64` poderia receber conteúdo malicioso tentando explorar o processamento no backend.

7.3 Repudiation - Repúdio

7.3.1 - Ausência de rastreabilidade de ações

Sem logs, um usuário poderia negar ter realizado ações como excluir a conta ou revogar consentimento.

7.4 Information Disclosure - Vazamento de Informação

7.4.1 - Retorno de dados sensíveis para o cliente

Sem serialização adequada, campos como `password` e `two_factor_secret` poderiam ser retornados nas respostas da API.

7.4.2 - Dados sensíveis armazenados sem criptografia

Informações como nome e segredo do 2FA poderiam estar em texto claro no banco de dados.

7.4.3 - API Key do Gemini exposta

A chave da API enviada pelo frontend via cabeçalho poderia ser capturada sem TLS.

7.5 Denial of Service - Negação de Serviço

7.5.1 - Flood de requisições no login

Muitas requisições simultâneas no endpoint de autenticação poderiam derrubar o servidor.

7.5.2 - Exaustão da cota da API Gemini

Sem limite, um usuário poderia esgotar a cota da API de análise de imagens rapidamente.

7.6 Elevation of Privilege - Elevação de Privilégio

7.6.1 - Acesso a rotas protegidas sem autenticação

Rotas que exigem login poderiam ser acessadas sem token caso não houvesse middleware de proteção.

7.6.2 - Token forjado com ID de outro usuário

Um token manipulado poderia tentar acessar dados de outros usuários se a assinatura não fosse verificada.

8. Associação Risco × Contramedida

Ameaça	Risco	Contramedida
Força bruta no login	Crítico	Bloqueio após 5 tentativas, atraso de 700ms e rate limiting
Reutilização de JWT após logout	Alto	Blacklist de tokens invalidados
Falsificação do token 2FA	Médio	Token com propósito dedicado e expiração de 5 minutos
Interceptação de dados em trânsito	Crítico	Redirecionamento forçado para HTTPS em produção
Adulteração de JWT	Alto	Assinatura HMAC-SHA256 com segredos distintos para cada tipo de token
Injeção via campo de imagem	Médio	Validação e normalização do payload antes do processamento
Rastreabilidade de ações	Médio	Log de auditoria registrando todas as ações sensíveis
Retorno de dados sensíveis no response	Alto	Serialização que exclui campos como password e two_factor_secret
Dados sem criptografia em repouso	Alto	AES-256-CBC para dados sensíveis e bcrypt para senhas

API Key do Gemini exposta	Alto	Transmissão via HTTPS e validação antes de qualquer uso
Flood de requisições	Crítico	Rate limiting global de 200 req/15min e 5 req/15min no login
Exaustão da cota do Gemini	Alto	Limite de 5 análises por minuto por usuário autenticado
Acesso a rotas sem autenticação	Crítico	Middleware de autenticação obrigatório em todas as rotas protegidas
Token com ID adulterado	Alto	Verificação criptográfica da assinatura JWT em toda requisição

9. Testes de Segurança Realizados

9.1 Testes Automatizados

O projeto utiliza Jest e Supertest no backend e Vitest com Testing Library no frontend.

9.1.1 - Verificação de saúde da API

Faz uma requisição para /health e confirma que o servidor está no ar e respondendo com status 200.

```
// src/tests/health.test.js

it("should return 200", async () => {
  const res = await request(app).get("/health")
  expect(res.statusCode).toBe(200)
  expect(res.body.ok).toBe(true)
})
```

9.1.2 - Rejeição de credenciais inválidas

Tenta fazer login com e-mail e senha errados e verifica que o sistema retorna um status de erro (400 ou superior).

```
// src/tests/auth.test.js

it("should fail with wrong credentials", async () => {

  const res = await request(app)

    .post("/api/auth/login")

    .send({ email: "wrong@test.com", password: "123" })

  expect(res.statusCode).toBeGreaterThanOrEqual(400)

})
```

9.1.3 - Renderização do formulário de login

Renderiza a página de login em ambiente de teste e confirma que os elementos da tela estão sendo exibidos corretamente.

```
// src/test/Login.test.jsx

it("should render login form", () => {

  renderWithProviders(<Login />)

  expect(screen.getByText(/login/i)).toBeInTheDocument()

})
```

9.2 Testes Manuais

9.2.1 - Bloqueio por força bruta

Foram enviadas 6 tentativas consecutivas de login com senha incorreta para um e-mail válido. A partir da 5ª tentativa, o sistema retornou status 401 com a mensagem "Conta temporariamente bloqueada por excesso de tentativas. Tente novamente mais tarde." e o campo `locked_until` foi preenchido no banco de dados.

9.2.2 - Acesso a rota protegida sem token

Requisição GET `/api/auth/me` realizada sem o cabeçalho `Authorization`. O servidor retornou status 401 com `{ "error": "Token não informado" }`.

9.2.3 - Uso de token após logout

Após realizar login, o `accessToken` obtido foi utilizado normalmente. Em seguida, foi feito logout via POST `/api/auth/logout`. Ao tentar usar o mesmo token novamente em GET `/api/auth/me`, o servidor retornou 401 com `{ "error": "Token inválido" }`, confirmando que a blacklist funcionou corretamente.

9.2.4 - Cadastro com senha fraca

Tentativa de registro com senha "123456", que não atende aos requisitos mínimos. O servidor retornou status 400 com mensagem descrevendo os requisitos de complexidade exigidos.

9.2.5 - Rate limiting na análise de imagem

Foram enviadas 6 requisições autenticadas seguidas para POST `/api/ai/analyze-image`. Na 6ª requisição, o servidor retornou status 429 com `{ "error": "Limite de análises atingido. Aguarde 1 minuto." }`.

9.2.6 - Fluxo 2FA com token expirado

Com 2FA ativo na conta, foi iniciado o login normalmente. O sistema retornou `requiresTwoFactor: true` e um `twoFactorToken`. Após aguardar mais de 5 minutos, o token intermediário foi enviado novamente e o servidor retornou erro de token inválido, confirmando a expiração configurada.

9.2.7 - Injeção NoSQL no campo de e-mail

Foi enviado o payload { "email": { "\$gt": "" }, "password": "qualquer" } no endpoint de login. A validação isValidEmail em auth.service.ts rejeitou o valor por não ser uma string válida, retornando 401 sem executar consulta indevida no banco.

9.2.8 - Exportação de dados sem campos sensíveis

Com usuário autenticado, foi feita requisição GET /api/user/export. O retorno continha apenas id, email, firstName, lastName, twoFactorEnabled, consent, consentDate e createdAt. Campos como password e two_factor_secret não apareceram na resposta, confirmando o comportamento da função serializeUser.

10. Resultados dos Testes Documentados

Teste	O que foi verificado	Resultado
Automatizado - Saúde da API	Servidor responde corretamente	Aprovado
Automatizado - Credenciais inválidas	Sistema rejeita login errado	Aprovado
Automatizado - Formulário de login	Interface renderiza corretamente	Aprovado
Manual - Força bruta	Bloqueio após 5 tentativas	Aprovado
Manual - Rota sem token	Rejeição sem autenticação	Aprovado
Manual - Token após logout	Blacklist funcionando	Aprovado
Manual - Senha fraca no cadastro	Validação de complexidade	Aprovado
Manual - Rate limit análise de imagem	Limite de 5/min por usuário	Aprovado
Manual - Token 2FA expirado	Expiração em 5 minutos	Aprovado
Manual - Injeção NoSQL	Validação de tipo de entrada	Aprovado
Manual - Export sem dados sensíveis	Serialização segura	Aprovado

11. Conformidade com a LGPD

11.1 Dados Pessoais Coletados

Dado	Finalidade	Base Legal	Retenção
E-mail	Identificação, autenticação, comunicação	Consentimento (art. 7º, I)	Até exclusão da conta
Senha (hash bcrypt)	Autenticação	Consentimento	Até exclusão da conta
Nome e Sobrenome	Personalização de perfil	Consentimento	Até exclusão da conta
Segredo TOTP (base32)	Autenticação 2FA	Consentimento	Até desativação do 2FA ou exclusão
Endereço IP	Segurança, auditoria, anti-fraude	Legítimo interesse (art. 7º, IX)	Conforme política de retenção
User-Agent	Segurança, auditoria	Legítimo interesse	Conforme política de retenção
Data/hora das ações	Auditoria e segurança	Legítimo interesse	Conforme política de retenção
Histórico de login	Segurança da conta	Legítimo interesse	Conforme política de retenção

11.2 Funcionalidades LGPD Implementadas

- ☐ Consentimento explícito: o registro exige `consent: true`. Sem consentimento, o cadastro é rejeitado.
- ☐ Registro de consentimento: data e hora do aceite são armazenados (`consent_date`).
- ☐ Revogação do consentimento: endpoint POST `/api/users/revokeconsent` permite revogar o consentimento a qualquer momento.
- ☐ Exportação de dados: GET `/api/users/export` retorna todos os dados pessoais do titular.
- ☐ Exclusão de conta: DELETE `/api/users/` remove o usuário e revoga todos os tokens ativos. O log de auditoria do evento é preservado por obrigação legal.
- ☐ Políticas publicadas: páginas dedicadas para Política de Privacidade, Política de Retenção e Política de Segurança estão disponíveis publicamente no frontend.

11.3 Minimização de Dados

O sistema coleta apenas os dados estritamente necessários para as finalidades declaradas. Não são coletados dados de localização geográfica, dados de cartão de crédito, dados biométricos ou qualquer informação sensível além das listadas acima. O endereço IP é coletado exclusivamente para fins de segurança e auditoria.

12. Auditoria e Logs

12.1 Arquitetura da Trilha de Auditoria

Toda ação relevante no sistema é registrada assincronamente em uma coleção dedicada (AuditLog) no MongoDB. O módulo de auditoria é desacoplado da lógica de negócio, o `logService.write()` é chamado nos controllers, após o resultado de cada operação.

O log captura, além da ação em si, o IP de origem do cliente (suportando proxy X-Forwarded-For para ambientes com load balancer) e o User-Agent do navegador, permitindo análises forenses.

12.2 Catálogo de Ações Auditadas

Código da Ação	Evento	Dados Adicionais (metadata)
REGISTER_SUCCESS	Cadastro bem-sucedido	—
REGISTER_FAILED	Falha no cadastro	reason: motivo do erro
LOGIN_SUCCESS	Login bem-sucedido	—
LOGIN_FAILED	Falha no login	reason: motivo do erro
LOGIN_2FA_REQUIRED	Login pausado aguardando 2FA	—
LOGOUT_SUCCESS	Logout bem-sucedido	—
LOGOUT_FAILED	Falha no logout	reason

REFRESH_SUCCESS	Renovação de token bem-sucedida	—
REFRESH_FAILED	Falha na renovação de token	reason
2FA_SETUP_STARTED	Configuração do 2FA iniciada	—
2FA_ENABLED	2FA ativado com sucesso	—
2FA_ENABLE_FAILED	Falha ao ativar 2FA	reason
2FA_DISABLED	2FA desativado	—
2FA_DISABLE_FAILED	Falha ao desativar 2FA	reason
PASSWORD_RESET_REQUESTED	Solicitação de recuperação de senha	—
PASSWORD_RESET_REQUEST_FAILED	Falha na solicitação de recuperação	reason
PASSWORD_RESET_SUCCESS	Senha redefinida com sucesso	—
PASSWORD_RESET_FAILED	Falha na redefinição de senha	reason
PROFILE_UPDATED	Perfil atualizado	—
PROFILE_UPDATE_FAILED	Falha ao atualizar perfil	reason
DATA_EXPORT_SUCCESS	Exportação de dados (LGPD)	—
DATA_EXPORT_FAILED	Falha na exportação de dados	reason
CONSENT_REVOKED	Consentimento LGPD revogado	—
CONSENT_REVOKE_FAILED	Falha ao revogar consentimento	reason
ACCOUNT_DELETED	Conta excluída	—
ACCOUNT_DELETE_FAILED	Falha ao excluir conta	reason

12.3 Exemplo de Análise de Logs

A seguir, exemplos de consultas MongoDB para análise e monitoramento de segurança usando os logs de auditoria do sistema:

12.3.1 — LOGIN_FAILED

auditlogs

SecureImageValidator

validate_image_system

auditlogs

Documents

171

Aggregations

Schema

Indexes

1

Validation

{ action: { \$in: ["LOGIN_FAILED"] } }

ADD DATA

UPDATE

DELETE

EXPORT CODE

_id: ObjectId('6a03674042734d3001b247e0')

user_id: null

email: null

action: "LOGIN_FAILED"

ip: "179.125.19.99"

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, L..."

metadata: Object

created_at: 2026-05-12T17:45:36.844+00:00

__v: 0

_id: ObjectId('6a037b3f42734d3001b247e6')

user_id: null

email: "mariana@gamil.com"

action: "LOGIN_FAILED"

ip: "179.125.19.99"

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, L..."

metadata: Object

created_at: 2026-05-12T19:10:55.359+00:00

__v: 0

_id: ObjectId('6a037b4b42734d3001b247e7')

user_id: null

email: "mariana@gamil.com"

action: "LOGIN_FAILED"

ip: "179.125.19.99"

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, L..."

metadata: Object

created_at: 2026-05-12T19:11:07.839+00:00

__v: 0

ood

12.3.2 — LOGIN_SUCCESS

auditlogs

SecureImageValidator

validate_image_system

auditlogs

Documents

171

Aggregations

Schema

Indexes

1

Validation

{ action: { \$in: ["LOGIN_SUCCESS"] } }

ADD DATA

UPDATE

DELETE

EXPORT CODE

_id: ObjectId('69fd363b40499ed0433e175e')

user_id: ObjectId('69fd2e249a64147b50d92045')

email: "guilhermeap.macedo20@gmail.com"

action: "LOGIN_SUCCESS"

ip: "177.63.205.242"

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, L..."

metadata: null

created_at: 2026-05-08T01:02:51.536+00:00

__v: 0

_id: ObjectId('69fd385f40499ed0433e1762')

user_id: ObjectId('69fd2e249a64147b50d92045')

email: "guilhermeap.macedo20@gmail.com"

action: "LOGIN_SUCCESS"

ip: "177.63.205.242"

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, L..."

metadata: null

created_at: 2026-05-08T01:11:59.187+00:00

__v: 0

_id: ObjectId('69fdcf40499ed0433e1767')

user_id: ObjectId('69fd2e249a64147b50d92045')

email: "guilhermeap.macedo20@gmail.com"

action: "LOGIN_SUCCESS"

ip: "177.63.205.242"

user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, L..."

metadata: null

created_at: 2026-05-08T15:16:55.733+00:00

__v: 0

12.3.3 — 2FA

SecureImageValidator > validate_image_system > auditlogs

Documents 171 Aggregations Schema Indexes 1 Validation

{ action: { \$in: ["2FA_ENABLED", "2FA_DISABLED"] } }

ADD DATA UPDATE DELETE EXPORT CODE ?

```
_id: ObjectId('6a10e2adffc72956955c021b')
user_id: ObjectId('6a0531c77952843ac571661e')
email: "guilhermeap.macedo20@gmail.com"
action: "2FA_ENABLED"
ip: "200.246.78.2"
user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l..."
metadata: null
created_at: 2026-05-22T23:11:41.983+00:00
__v: 0
```

```
_id: ObjectId('6a10e31bffc72956955c021c')
user_id: ObjectId('6a0531c77952843ac571661e')
email: "guilhermeap.macedo20@gmail.com"
action: "2FA_DISABLED"
ip: "200.246.78.2"
user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l..."
metadata: null
created_at: 2026-05-22T23:13:31.041+00:00
__v: 0
```

12.3.4 — BUSCA POR EMAIL

SecureImageValidator > validate_image_system > auditlogs

Documents 171 Aggregations Schema Indexes 1 Validation

{ email: { \$in: ["guilhermeap.macedo20@gmail.com"] } }

ADD DATA UPDATE DELETE EXPORT CODE ?

```
_id: ObjectId('69fd2e0f9a64147b50d92044')
user_id: ObjectId('69fd2d3ec701f4f8e5825b76')
email: "guilhermeap.macedo20@gmail.com"
action: "LOGOUT_SUCCESS"
ip: "::1"
user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l..."
metadata: null
created_at: 2026-05-08T00:27:59.623+00:00
__v: 0
```

```
_id: ObjectId('69fd2e249a64147b50d92046')
user_id: ObjectId('69fd2e249a64147b50d92045')
email: "guilhermeap.macedo20@gmail.com"
action: "REGISTER_SUCCESS"
ip: "::1"
user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l..."
metadata: null
created_at: 2026-05-08T00:28:20.503+00:00
__v: 0
```

```
_id: ObjectId('69fd362d40499ed0433e175c')
user_id: null
email: "guilhermeap.macedo20@gmail.com"
action: "REGISTER_FAILED"
ip: "177.63.205.242"
user_agent: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, l..."
metadata: Object
created_at: 2026-05-08T01:03:37.335+00:00
__v: 0
```

12.4 Proteção contra Alteração dos Logs

A proteção da integridade dos logs é garantida por quatro mecanismos:

- ❑ O módulo de auditoria só expõe o método `write()` — não há implementação de `update` ou `delete` nos logs.
- ❑ O MongoDB Atlas pode ser configurado com usuário de banco com privilégios somente de escrita e leitura na coleção de logs, bloqueando DDL e delete.
- ❑ Os logs não são expostos por nenhum endpoint público da API — não há rota de consulta ou exclusão de logs.
- ❑ O campo `created_at` é gerado automaticamente pelo Mongoose com timestamps, sem campo `updatedAt`, reforçando a imutabilidade temporal.

13. Normas técnicas

OWASP Top 10

O OWASP Top 10 lista os principais riscos de segurança em aplicações web. O projeto trata alguns deles, como o Broken Access Control, resolvendo usando um middleware de autenticação e blacklist de tokens; Cryptographic Failures, solucionando através de `bcrypt` para senhas, AES-256-CBC para dados em repouso; Injection, protegendo através da validação de entrada e uso do Mongoose e Identification and Authentication Failures, tratado usando senha forte obrigatória, bloqueio de conta, 2FA, expiração de tokens

LGPD - Lei nº 13.709/2018

A Lei Geral de Proteção de Dados foi base para a implementação das funcionalidades de privacidade do sistema, incluindo exportação de dados, exclusão de conta e revogação de consentimento, que correspondem aos direitos dos titulares previstos no Artigo 18 da lei.

CONCLUSÃO

A aplicação desenvolvida alcançou os objetivos propostos ao oferecer um ambiente seguro para análise de imagens potencialmente geradas por inteligência artificial, aliado a práticas de autenticação forte, controle de sessões e proteção contra ameaças comuns em aplicações web. Recursos como autenticação em dois fatores (2FA), utilização de JWT, criptografia com bcrypt, rate limiting e controle de acessos contribuíram significativamente para o fortalecimento da segurança do sistema.

Além disso, o projeto buscou atender princípios relacionados à privacidade e conformidade com a Lei Geral de Proteção de Dados (LGPD), incorporando funcionalidades como consentimento do usuário, exportação de dados pessoais, revogação de consentimento e exclusão de conta. Essas práticas reforçam a importância da responsabilidade no tratamento de dados e evidenciam a preocupação com a transparência e a segurança das informações armazenadas.

Do ponto de vista acadêmico e técnico, o desenvolvimento da aplicação proporcionou experiências práticas relacionadas à arquitetura em camadas, integração entre frontend e backend, implementação de APIs REST, autenticação segura e organização de sistemas escaláveis. O projeto também possibilitou maior compreensão sobre os desafios atuais envolvendo conteúdos gerados por inteligência artificial e os impactos da segurança da informação no contexto digital.

Por fim, conclui-se que o sistema desenvolvido apresenta uma solução funcional, moderna e alinhada às boas práticas de desenvolvimento seguro, demonstrando o potencial de aplicações que unem inteligência artificial, autenticação forte e proteção de dados para promover maior confiabilidade e segurança no ambiente digital.

REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: seção 1, Brasília, DF, 15 ago. 2018. Disponível em: <https://www.planalto.gov.br>. Acesso em: 23 abr. 2026.

ISO/IEC 27001. Information technology — Security techniques — Information security management systems — Requirements. Geneva: ISO, 2013.

OWASP. OWASP Top Ten 2021. Open Web Application Security Project, 2021. Disponível em: <https://owasp.org/Top10>

REPOSITÓRIOS

Backend: <https://github.com/guilhermemacedo20/image-validator-backend>

Frontend: <https://github.com/guilhermemacedo20/image-validator-frontend>

Site: <https://image-validator-frontend-cyan.vercel.app>